Name: _____

Class: _____

Mr. Rawson • GCSE Computer Science Year 10 2026

Hexadecimal Practice 2

Eight cards. No pens, no arithmetic, no subtraction.

Start here. Cut the eight place-value cards off the **card sheet**.

The back of every card is blank, and that is the point. A card **face up** means that place value is switched **on** — a `1`. A card turned **blank side up** means **off** — a `0`. The grey number *above* each cell tells you which card it is, so you never lose your place.

The whole method, in one line: add up the place values that are face up. Nothing is subtracted and nothing is written down until you have the answer.

First job · build your reference strip

four cards, sixteen numbers — and you keep it

Take just **four** cards — `8`, `4`, `2`, `1` — and make every number from **0 to 15**. **Denary is given along the top**; fill in the hex underneath it and the binary below that. One column of each strip is done for you.

Keep this strip. You will use it on every page that follows, for the **quiz on Thu 17 Sept** and for the **test on Tue 6 Oct** — and you will trust it, because you built it.

Denary	0	1	2	3	4	5	6	7
Hex	0							
Binary	0000							

Denary	8	9	10	11	12	13	14	15
Hex			A					
Binary			1010					

Four cards run out at 15 — the whole reason hex needs letters. Sixteen values, and only ten digits to write them with.

One binary number. We are going to read it two ways.

00110100

Part A · Binary to denary

two steps

Step 1 Set all eight cards to match the number. **Face up** for a `1`, **blank side up** for a `0`.

Step 2 **Add up the place values that are face up.**

128	64	32	16	8	4	2	1

$$32 + 16 + 4 \;=\; \mathbf{52}$$

Now you. `01101010` = _____ `10000011` = _____

Part B · The same number, in hexadecimal

four steps

Step 1 **Split the byte into two nibbles** — four bits each. Slide the right-hand four away from the left-hand four.

0011 **0100**

Step 2 **Work out the denary value of each nibble**, using the four-bit place values. **Read each mat on its own** — a nibble reads `8 4 2 1` whichever half it came from.

8	4	2	1

$2 + 1 = \mathbf{3}$

the gap

8	4	2	1

$4 = \mathbf{4}$

Step 3 **Change each denary value into its hex digit** — look it up on the strip you built.

$3 \rightarrow 3$ $4 \rightarrow 4$

Step 4 **Read the hex digits from left to right.**

$$\mathbf{00110100_2} \;=\; \mathbf{34_{16}}$$

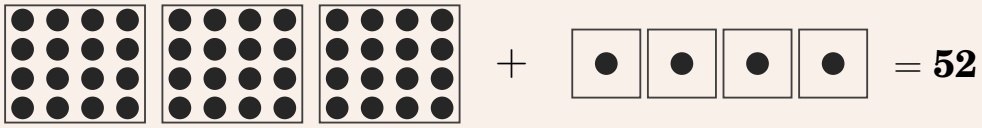
And it is still 52. `34` in hex *is* 52 in denary — check it: $3 \times 16 + 4 = 52$. One pattern of eight cards, written down two ways. **Hexadecimal is just where you put the gap.**

Part C · Hexadecimal back to denary

On page 2 you split `0011 0100` into two nibbles and read **3** and **4**. **But the left nibble was never just 3.** It was **3 sixteens**. Cut out the sixteen cards on the lower half of the **card sheet**, take a set of singles, and come back.

In a hex pair, the left digit counts *cards* and the right digit counts *dots*.

34 — a number you have already met



3 cards = 48 **4 singles**

Same number, third time. On page 2 it was `0011 0100`, $32+16+4=52$. In hex, `34`. On the desk, three cards and four singles. **Nothing changed but how you are looking at it.**

Your turn. Build each one, then fill the box in. **The `16` column is your cards; the `1` column is your singles** — same box as the nibble mats, place values on top, values underneath.

4B

	16	1
Hex	4	B
Value		

Total = _____

7C

	16	1
Hex	7	C
Value		

Total = _____

DE

	16	1
Hex	D	E
Value		

Total = _____

You met `DE` on page 2. If the desk and the byte strip disagree, finding out which is wrong is the useful part.

Count out sixteen singles.

Now put them beside **one card**. They are worth exactly the same. **Sixteen singles *are* one card** — so swap them, and the right-hand digit drops to nought while the left-hand one goes up by one.

That swap is the only thing place value has ever meant.

Then stop using them.

You can build *every* byte there is — `FF` is 15 cards and 15 singles. But laying `DE` out takes a minute, and working it out takes five seconds: $13 \times 16 = 208$, plus **14**, is **222**.

The cards are how you learn to trust the arithmetic. Then you leave them behind.

Now put all sixteen cards on the desk at once. That is $16 \times 16 =$ **256** dots — but a byte's biggest number is **255**, because `FF` is **15 cards and 15 singles** $= 255$. **The sixteenth card is one too many.** A byte counts **0 to 255** — 256 values, and you have just laid every one of them out. **Next: a colour code is three byte strips side by side.**